COURSE BUILD SYSTEM

Slide layout reference

Each slide shows one layout, drawn from slides.md with illustrative content

23 example slides

Every layout and slide type

Today's plan

09:00	Welcome, prediction contracts, and evidence	Lecture
09:45	Splits, leakage, and baselines	Lecture
10:30	Break	Break
10:45	Lab 1: baseline and split design	Lab
11:30	Linear models and regularization	Lecture
12:30	Lunch	Lunch
13:30	Classification: KNN, SVM, trees	Lecture
14:45	Break	Break
15:00	Lab 2: a defensible comparison	Lab
16:20	Evidence review and exit check	Discussion

LEARNING OUTCOMES

What you will be able to do

By the end of the day you will be able to:

1. Turn an energy problem into a target, prediction unit, and evaluation design.
2. Fit and compare linear regression, KNN, SVM, and decision trees.
3. Explain overfitting using residuals and validation results.
4. Build a preprocessing pipeline and defend its final evaluation.

PART 1 OF 1

01

Problem framing and evidence

A model is only as useful as the question it answers.

- Prediction contracts
- Splits that match deployment
- Leakage and baselines

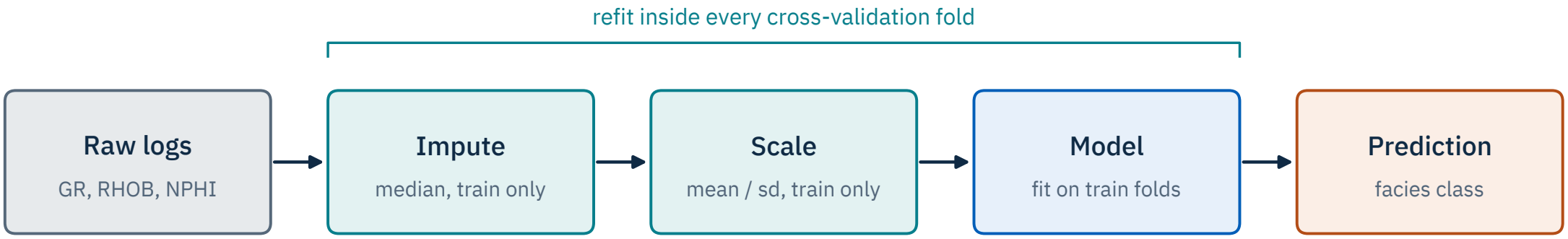
Baselines make improvement measurable

- Regression: compare with the training-target mean or a domain forecast.
- Classification: compare with a majority-class or simple linear classifier.
  - Use `DummyClassifier(strategy="most_frequent")` as the floor.
- Use the same evaluation rows and metric for every candidate.
- Record runtime and failure cases alongside the score.

◆ A model that cannot beat a persistence forecast has not yet earned its complexity.

Preprocessing belongs inside the pipeline

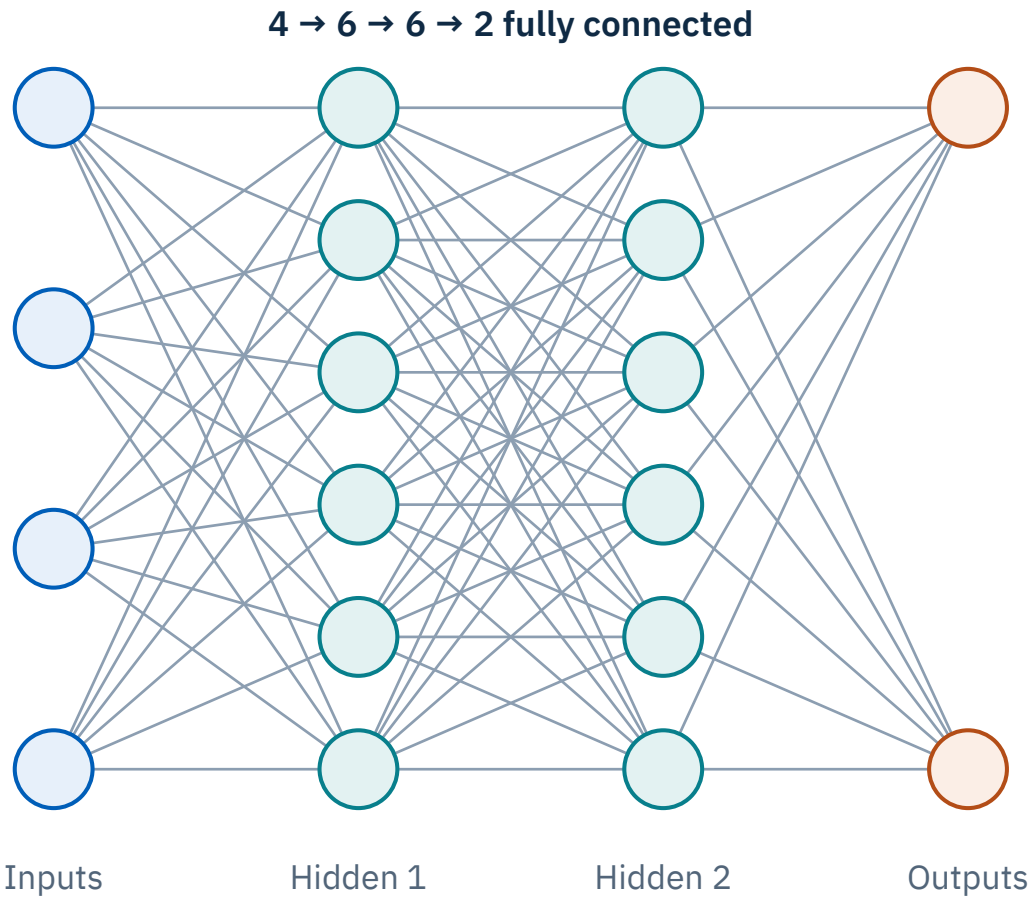
Every fitted step — imputation, scaling, selection — learns only from the training fold.



Schematic. Steps inside the bracket are refitted on each training fold during cross-validation.

A network is a stack of simple functions

- Each hidden unit computes a weighted sum and applies a nonlinearity.
- Layers compose: the output of one is the input to the next.
- Width and depth set capacity; data and regularization decide whether it is used well.

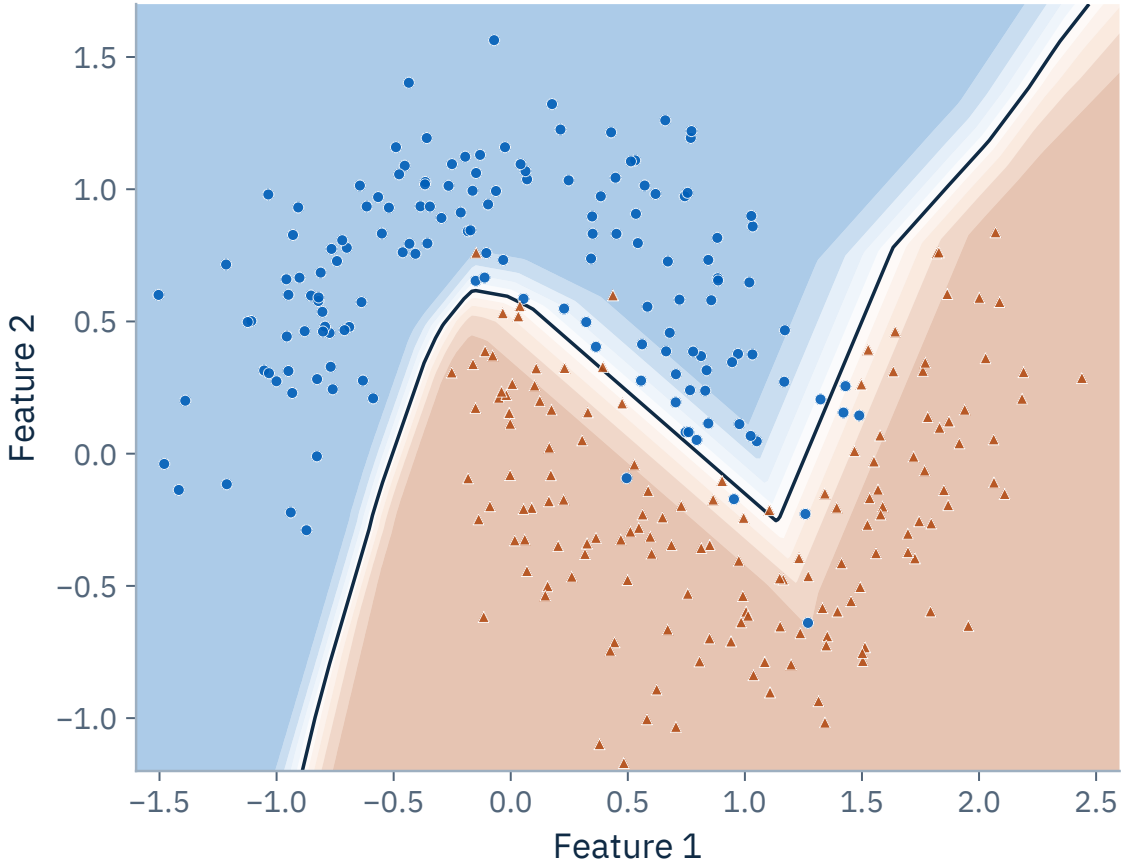


◆ Parameters: 4·6+6 + 6·6+6 + 6·2+2 = 86.

Gradient descent updates every weight

$$w_{t+1} = w_t - \eta \frac{\partial L}{\partial w}$$

- η is the learning rate; too large diverges, too small stalls.
- The gradient points uphill, so we step against it.
- Mini-batches estimate the gradient cheaply and add useful noise.



MLP (two hidden layers of 16 units) fitted to synthetic moons. Shading shows predicted probability.

Bagging and boosting use different training logic

Bagging

- Trains models **in parallel** on bootstrap samples.
- Averages to reduce variance.
- Random forest adds feature subsampling.

Boosting

- Trains models **in sequence**.
- Each model fits what the ensemble still gets wrong.
- Reduces bias; needs early stopping.

Stacking

- Learns how to **combine** different models.
- Needs out-of-fold predictions to avoid leakage.

◆ All three need the same honest validation design as a single model.

Scaling: inside or outside the fold?

✗ Fit on all rows, then split

- Validation statistics leak into training.
- Scores are optimistic, sometimes badly.

vs

✓ Fit inside each training fold

- Validation rows stay unseen.
- Scores estimate deployment performance.

The modelling workflow

A project moves through five stages, and evidence flows back from each one.

1. **Frame** — target, prediction time, unit of analysis
2. **Audit** — units, missingness, leakage risks
3. **Baseline** — the simplest defensible model
4. **Compare** — controlled experiments on a fixed split
5. **Deploy** — monitor drift and re-validate

◆ Most project failures start at step 1, not step 4.

PRACTICE

4 min · Pairs

Accuracy can hide the minority class

Using TN = 85, FP = 5, FN = 3, TP = 7:

1. Calculate accuracy, event precision, and event recall.
2. Compare with a model that always predicts no event.
3. Which error would matter most for the proposed use?

The always-negative model looks excellent

Same 100 wells, 10 real events. A model that never raises an alert:

90%

accuracy

0%

event recall

0

useful alerts

- Report per-class recall and precision whenever events are rare.

Regression metrics answer different questions

Metric	Interpretation	Limitation
MAE	Average absolute error, in target units	Treats all error sizes linearly
RMSE	Error in target units, emphasizes large misses	Sensitive to outliers
R^2	Relative to an evaluation-set mean reference	Can be negative

Report errors by well or operating regime as well as overall.

Grouped tuning respects well boundaries

```python
from sklearn.model_selection import GroupKFold, GridSearchCV

search = GridSearchCV(
    model,
    {"svc__C": [0.1, 1, 10], "svc__gamma": [0.01, 0.1]},
    cv=GroupKFold(n_splits=3), scoring="f1_macro",
)
search.fit(X_train, y_train, groups=well_train)
print(search.best_params_)
```

OUTPUT

```
{'svc__C': 1, 'svc__gamma': 0.1}
```

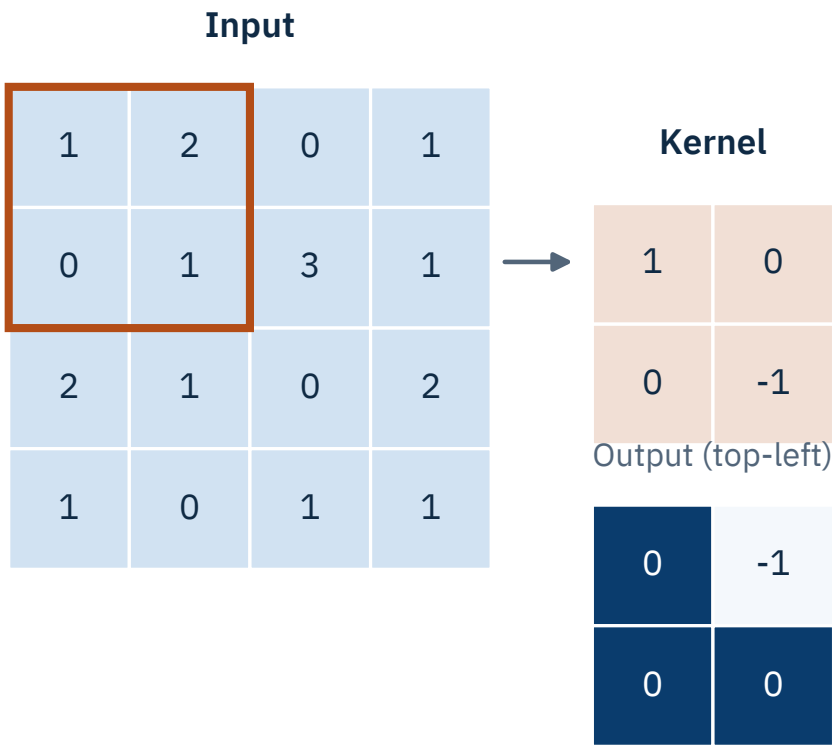
Reserve the final test wells before running this search.

A pipeline in five lines

```
model = make_pipeline(
    SimpleImputer(strategy="median"),
    StandardScaler(),
    SVC(kernel="rbf", C=1.0),
)
scores = cross_val_score(model, X, y, cv=cv)
```

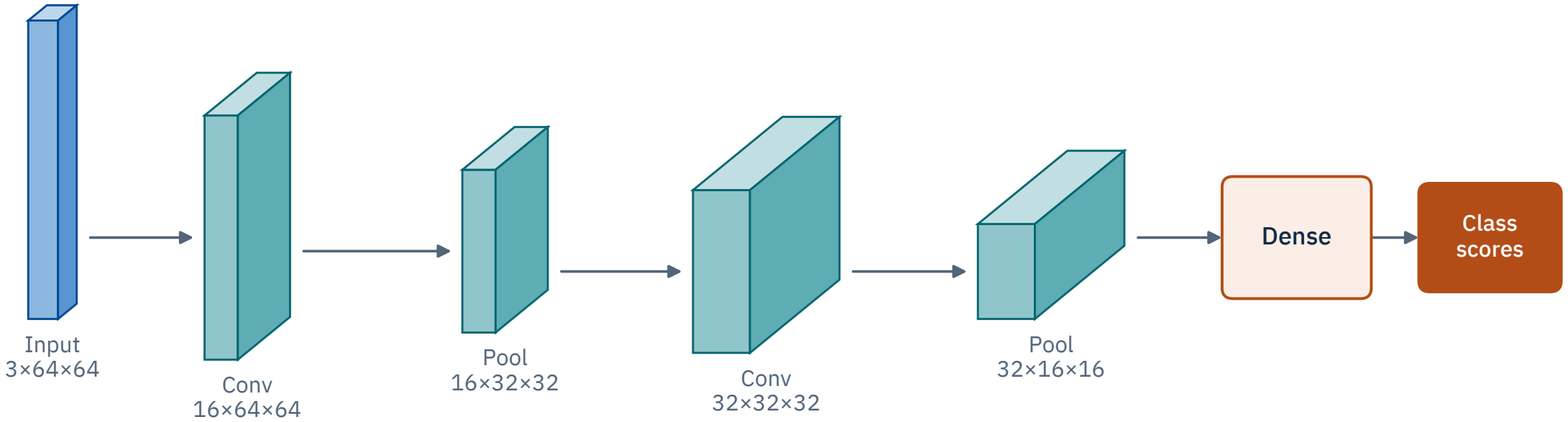
- `make_pipeline` chains fitted steps.
- Every CV fit re-learns the imputer and scaler.
- Categorical columns need a `ColumnTransformer`.

Convolution by numbers



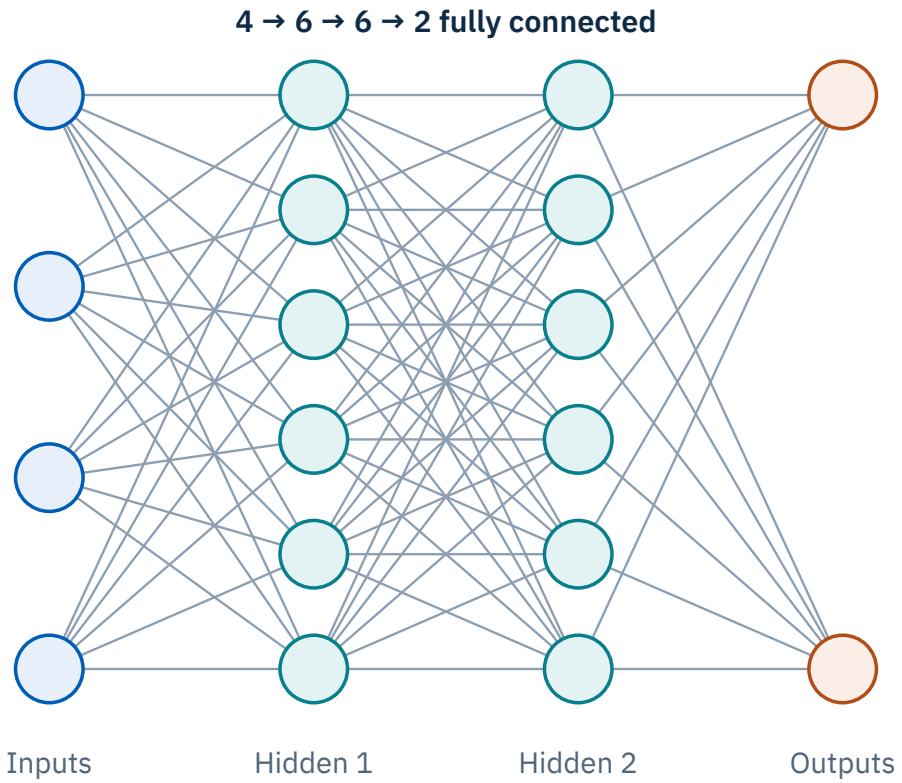
- Slide the 2×2 kernel over the input.
- Multiply element-wise and sum.
- Top-left: $1\cdot1+2\cdot0+0\cdot0+1\cdot(-1)=0$.

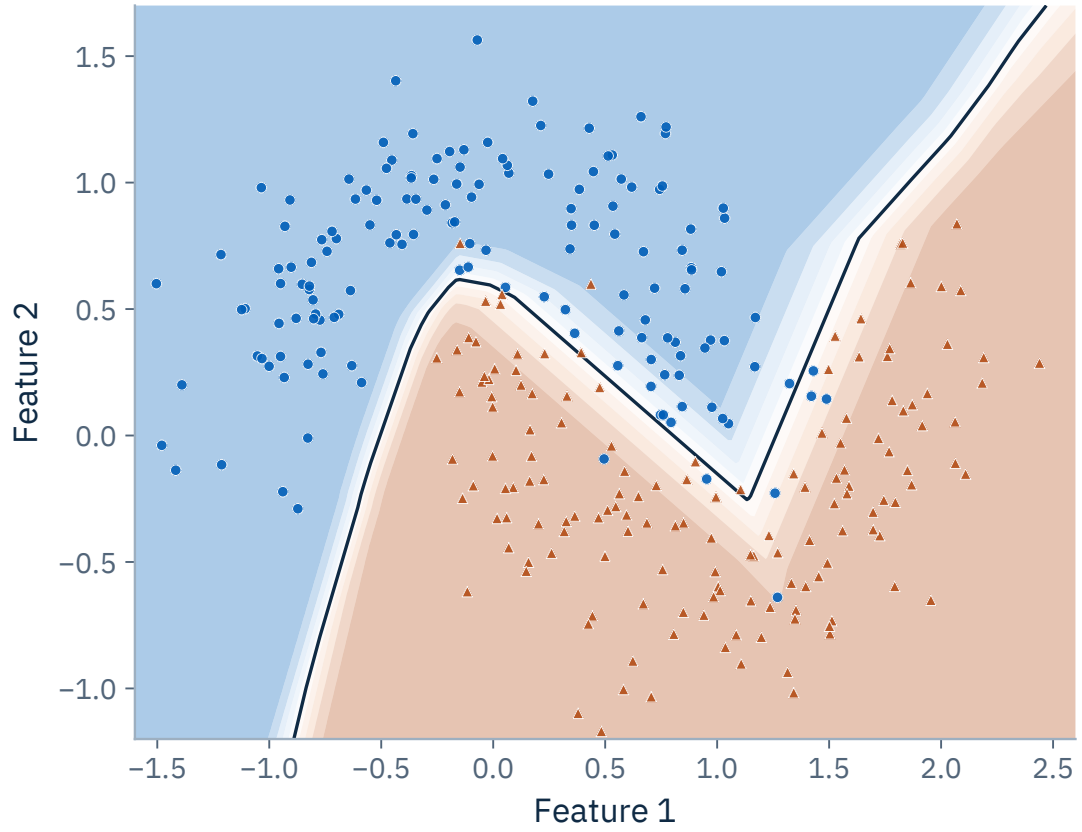
A CNN turns pixels into class scores



Schematic. Spatial size shrinks while the number of channels grows.

Architecture and behaviour side by side





Left: schematic of a small multilayer perceptron. Right: a two-hidden-layer MLP fitted to synthetic moons (seed 0). Pair layouts need HALF-size figures.

Complexity should be earned, not assumed

A simple model you can explain beats a complex model you cannot validate.

Prefer the simplest model that meets the requirement on honest validation data.

LABORATORY

80 min

Laboratory: a defensible comparison

- [ ] Load the data and write the prediction contract in the first cell.
- [ ] Fit one baseline and two candidates on a fixed split.
- [ ] Plot residuals or a confusion matrix for the best candidate.
- [ ] Write down one failure case and a hypothesis for it.

NOTEBOOK

`exercises/01_supervised_learning_basics/Supervised_Learning_Basics_Student.ipynb`

EVIDENCE TO SUBMIT

- A score table with the baseline first
- One residual or confusion plot
- A failure case with a hypothesis

Key takeaways

- ✓ The split must answer the deployment question.
- ✓ Fit every preprocessing step inside the training fold.
- ✓ A baseline makes improvement measurable.
- ✓ Capacity is controlled by regularization and validated, not guessed.

References and further study

- scikit-learn User Guide: linear models, neighbours, SVM, decision trees.
- scikit-learn: common pitfalls and cross-validation.
- Hastie, Tibshirani, Friedman: *The Elements of Statistical Learning*, chapters 2, 3, 9, and 12.